

AIRBUS

Direction des Systèmes d'Information, Infrastructure et Sécurité Réseau

Partie 1 : automatisation de l'infrastructure

État d'avancement : Terraform et Ansible

En deux mots

La partie 1 du cahier des charges est désormais automatisée de bout en bout. **Terraform** crée les six machines et les sept réseaux du laboratoire. **Ansible** installe et configure les services métiers, la supervision et les deux pare-feux. L'ensemble est rejouable : détruire puis reconstruire le laboratoire redonne exactement la même infrastructure, avec la même configuration, sans intervention manuelle.

1. Vue d'ensemble

Le projet s'appuie sur trois outils, chacun avec un périmètre précis. Ce découpage n'est pas cosmétique : il découle d'une limite technique de cloud-init, qui ne s'exécute qu'une seule fois, au premier démarrage de la machine.

LES TROIS COUCHES D'AUTOMATISATION

Qui fait quoi, et ce que l'on peut rejouer

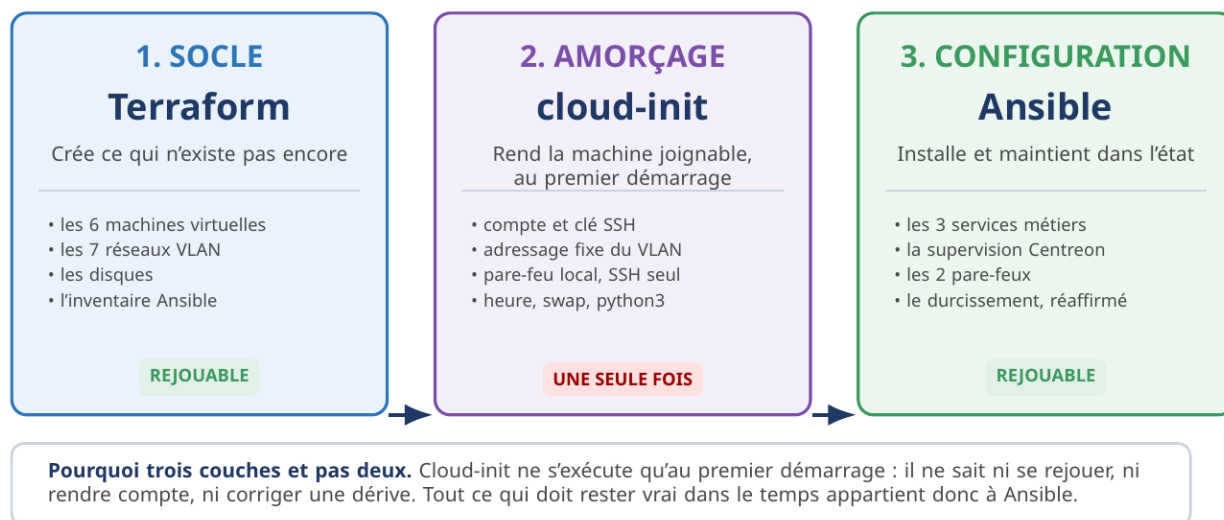


FIGURE 1. Les trois couches d'automatisation

Une première version confiait l'installation des services à cloud-init. Cette approche a été abandonnée : cloud-init ne sait ni se rejouer, ni rendre compte de ce qu'il a fait, ni corriger une configuration qui aurait dérivé. La moindre modification obligerait à détruire puis recréer la machine. Il a donc été ramené à son seul rôle d'amorçage.

TABLEAU 1. Volumétrie du projet

Couche	Lignes	Unités	Contenu
Terraform	1 100	3 modules	reseau, serveur, parefeu
Ansible	1 500	7 rôles	84 tâches réparties par service
Total	2 600		Commentaires : 6 % du code

2. Terraform : le socle

La cible retenue est KVM et libvirt, ce qui correspond au scénario B du cahier des charges : une virtualisation locale, sans cloud public. C'est cette architecture qui sera déployée sur la machine Intel fournie par l'entreprise.

ARCHITECTURE DU CODE : INFRASTRUCTURE TERRAFORM

Déploiement d'une infrastructure KVM avec libvirt

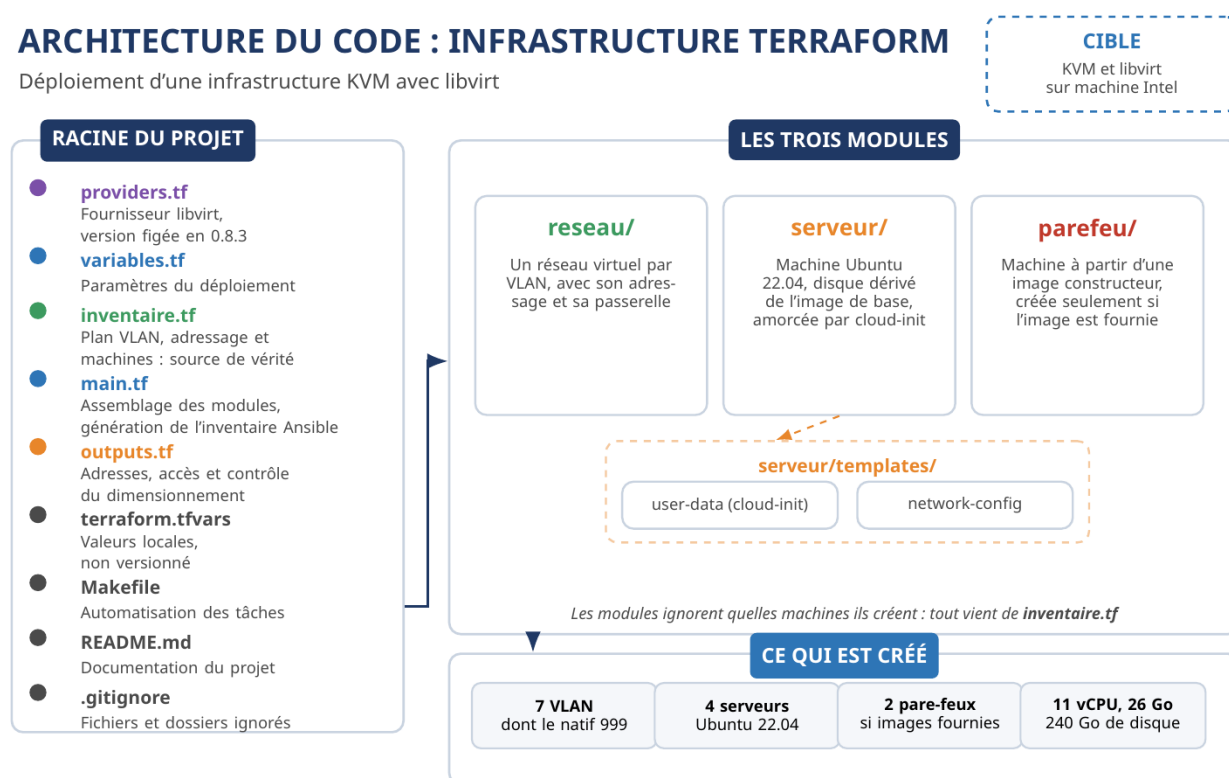


FIGURE 2. Architecture du code : infrastructure Terraform

2.1 Un inventaire centralisé

Le fichier `inventaire.tf` est la source de vérité de l'infrastructure. Il reprend les tableaux 6, 7 et 8 du cahier des charges : plan VLAN, adressage et machines.

L'objectif est de rassembler toutes les informations d'infrastructure au même endroit. Ajouter un serveur ou changer une adresse se fait uniquement là. Les modules, eux, ignorent quelles machines ils créent : on leur transmet des valeurs. Cette séparation facilitera aussi une migration vers le cloud, puisque la logique de déploiement reste indépendante de l'inventaire.

2.2 Ce qui est automatisé

- les 7 VLAN, y compris le VLAN natif 999 qui n'a pas d'adressage ;
- les 4 serveurs Ubuntu 22.04, avec adressage fixe ;
- les 2 pare-feux, lorsqu'une image constructeur est fournie ;
- l'inventaire Ansible, écrit automatiquement après la création des machines.

Pour les pare-feux, les images sont optionnelles. Si aucune n'est renseignée, les autres machines se déploient normalement et les sorties Terraform indiquent simplement que les pare-feux ne sont pas déployés.

Version du fournisseur libvirt volontairement figée

La version 0.9 du fournisseur libvirt repose sur une API entièrement différente et encore peu documentée, incompatible avec le code actuel. La version a donc été figée sur **libvirt 0.8.3**, la référence stable du projet. Sans cela, un prochain `terraform init` installerait automatiquement une version plus récente et casserait le déploiement.

3. Ansible : la configuration

Terraform crée les machines, il ne les configure pas. Ansible prend le relais pour installer les services, la supervision et les pare-feux, et surtout pour maintenir cette configuration dans le temps.

ARCHITECTURE DU CODE : CONFIGURATION ANSIBLE

Configuration des machines créées par Terraform

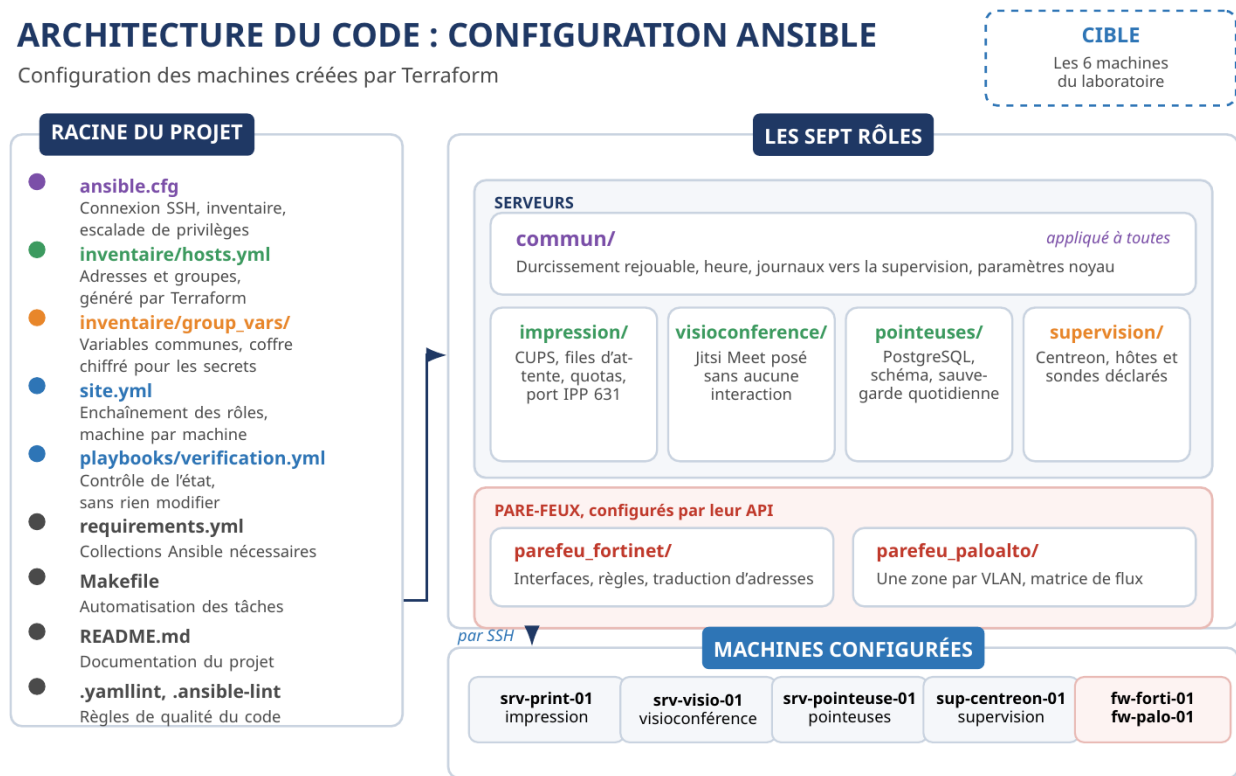


FIGURE 3. Architecture du code : configuration Ansible

3.1 Les sept rôles

TABLEAU 2. Rôles Ansible et périmètre de chacun

Rôle	Machine	Ce qu'il fait
commun	toutes	Durcissement rejouable, heure, journaux vers la supervision, paramètres noyau
impression	srv-print-01	CUPS, files d'attente, quotas, accès limité au VLAN des postes
visioconference	srv-visio-01	Jitsi Meet installé sans aucune interaction
pointeuses	srv-pointeuse-01	PostgreSQL, schéma de collecte, sauvegarde quotidienne, contrôle de l'horloge
supervision	sup-centreon-01	Centreon, réception des journaux, déclaration des machines et des sondes
parefeu_fortinet	fw-forti-01	Interfaces, règles, traduction d'adresses, export de la configuration
parefeu_paloalto	fw-palo-01	Une zone et une sous-interface par VLAN, matrice de flux, profils

3.2 Un inventaire généré, jamais écrit à la main

Quand on sépare la création de l'infrastructure de sa configuration, une erreur classique consiste à maintenir deux listes d'adresses, une par outil. Elles finissent toujours par diverger.

Ici, le fichier `ansible/inventaire/hosts.yml` est **généré par Terraform** à chaque déploiement, directement depuis `inventaire.tf`. Les groupes sont déduits d'un champ porté par chaque machine. Ajouter un serveur revient donc à ajouter une entrée dans l'inventaire Terraform : la machine est créée, et Ansible sait immédiatement quel rôle lui appliquer.

Un pare-feu n'apparaît dans l'inventaire que si son image a été fournie. Tant que ce n'est pas le cas, Ansible ne tente pas de le configurer.

3.3 Le durcissement, mais rejouable

Le durcissement demandé par **INF-06** est posé par `cloud-init` à la création de la machine, puis réaffirmé par le rôle commun à chaque exécution.

Cette redondance est volontaire. `Cloud-init` garantit qu'une machine neuve est conforme ; le rôle commun garantit qu'elle l'est encore trois semaines plus tard, même si quelqu'un a ouvert un port entre-temps. C'est ce qui permet de prouver la conformité à tout moment, et non seulement le jour de la création.

3.4 La supervision, qui satisfait réellement INF-04

L'exigence **INF-04** demande qu'une machine sous Centreon surveille toutes les autres. Installer Centreon ne suffit donc pas : encore faut-il que les machines y soient déclarées avec leurs sondes.

Le rôle procède en trois temps : installation depuis le dépôt officiel, réception des journaux des autres machines, puis déclaration des hôtes et des sondes par la ligne de commande Centreon plutôt que par l'interface web, afin que l'opération reste rejouable. Les sondes dépendent du rôle de chaque machine et reprennent le tableau des indicateurs du cahier des charges.

3.5 Les pare-feux, en deux temps

Les images FortiOS et PAN-OS n'acceptent pas `cloud-init` : ce sont des systèmes constructeur fermés. La configuration se fait donc en deux étapes.

L'amorçage, une seule fois, rend la machine joignable et licenciée. Palo Alto dispose d'un mécanisme officiel, un disque virtuel contenant `init-cfg.txt` et `bootstrap.xml`. FortiGate accepte une configuration déposée sur une image ISO attachée au premier démarrage.

La configuration passe ensuite par l'API du constructeur, autant de fois qu'on le souhaite. Les collections `fortinet.fortios` et `paloaltonetworks.panos` sont idempotentes : chaque exécution ramène le pare-feu à l'état décrit dans les fichiers de variables. La configuration obtenue est ensuite exportée dans le dépôt Git, ce qui répond à **INF-08** et **INF-10**.

Limites de la licence d'évaluation Fortinet

Elle plafonne la machine à trois interfaces, trois règles et trois routes. Le rôle vérifie cette limite avant d'appliquer quoi que ce soit et s'arrête avec un message clair si la configuration la dépasse, plutôt que de laisser l'API échouer.

3.6 Gestion des secrets

Aucun mot de passe ne figure en clair dans le dépôt. Les valeurs sensibles vivent dans un fichier chiffré par `ansible-vault`, qui peut être versionné sans risque. Seul le mot de passe du coffre ne l'est jamais.

4. Vérifications réalisées

Plusieurs contrôles ont été menés pour valider le code avant tout déploiement.

TABLEAU 3. Contrôles effectués sur les deux couches

Couche	Contrôle	Résultat
Terraform	terraform fmt	Conforme
Terraform	terraform validate	Success ! The configuration is valid.
Terraform	terraform plan	Va jusqu'au calcul des sorties, s'arrête uniquement faute de libvirt sur le poste actuel
Terraform	Rendu des cloud-init	YAML valide, 14 contrôles passés
Terraform	Inventaire	6 machines conformes, 11 vCPU, 26 Go, 240 Go
Ansible	yamllint	Conforme
Ansible	ansible-playbook --syntax-check	Conforme
Ansible	ansible-lint	0 échec, profil production, 5 étoiles sur 5
Ansible	Lecture de l'inventaire	6 groupes correctement construits
Les deux	Cohérence croisée	6 groupes déclarés, 7 rôles présents, aucun orphelin

L'analyse par ansible-lint a permis de corriger cinq points, dont **deux véritables erreurs** qui auraient échoué à l'exécution : le module postgresql_query n'accepte pas le paramètre path_to_script, et le module panos_ntp n'existe pas dans la collection Palo Alto. Sans cette analyse préalable, ces deux erreurs n'auraient été découvertes qu'en plein déploiement sur la machine cible.

5. Conformité aux exigences

TABLEAU 4. État des exigences INF-01 à INF-10

Réf.	Exigence	État
INF-02	Une machine par service métier	Conforme
INF-03	Ubuntu Server 22.04 LTS	Conforme
INF-04	Centreon surveille toutes les machines	Conforme , hôtes et sondes déclarés
INF-05	Plan VLAN et adressage validés	Conforme
INF-06	Durcissement des serveurs	Conforme , et rejouable
INF-07	Heure synchronisée par NTP	Conforme , serveurs et pare-feux
INF-08	Configurations versionnées dans Git	Conforme
INF-09	Création automatisée par Terraform	Conforme
INF-10	Sauvegarde avant modification	Conforme , export des pare-feux
INF-01	Deux pare-feux déployés	Code prêt , images constructeur à fournir

6. Ce qui reste à faire

Trois points restent ouverts, et il est plus honnête de les poser clairement.

Le code n'a pas encore tourné contre un véritable hyperviseur. Il est validé sur le plan syntaxique et cohérent d'un bout à l'autre, mais il n'a pas été éprouvé à l'exécution. Le poste actuel est en Apple Silicon avec 16 Go de mémoire, et libvirt n'y est pas disponible. Le premier déploiement réel se fera sur la machine Intel fournie par l'entreprise.

Les images des pare-feux restent à récupérer auprès de Fortinet et de Palo Alto. Tant qu'elles ne sont pas fournies, les serveurs se déploient normalement et les pare-feux sont simplement absents de l'inventaire.

Centreon s'arrêtera sur Ubuntu. L'éditeur ne publie de dépôt que pour Debian et la famille RHEL. Le rôle le détecte et s'arrête avec un message expliquant la solution déjà prévue au cahier des charges : basculer cette seule machine sous Debian 12.

7. Conclusion

L'infrastructure de la partie 1 est suffisamment avancée pour permettre un déploiement reproductible sur la future machine Intel. Terraform couvre le réseau, les machines virtuelles, l'adressage et l'amorçage. Ansible couvre l'installation des services, la supervision et la configuration des deux pare-feux.

Ce qui a changé récemment tient en trois points. Les services métiers ne sont plus seulement préparés mais réellement installés et configurés. La supervision déclare désormais les machines et leurs sondes, ce qui satisfait enfin **INF-04**. Enfin, les deux pare-feux disposent d'une configuration complète et rejouable, là où il n'y avait auparavant que la création de la machine.

Les éléments qui restent manuels correspondent aux composants dont la configuration dépend d'informations externes ou d'images constructeur : le domaine et le certificat TLS de Jitsi, les images des pare-feux, et la distribution attendue par Centreon.

L'objectif de la prochaine étape est de réaliser un déploiement complet sur la machine cible, afin de valider l'ensemble de l'architecture en conditions réelles.